Personalized Video Re-ranking via Reinforcement Learning with Criterion-Level Credit Assignment

Amit Roy[1*], **Bharat Bhargava**[1], **Jingru Cheng**[2], **Phanideep Gampa**[2], **Sheikh Sarwar**[2], **Chaosheng Dong**[2], **Haiyang Zhang**[2]
Purdue University[1], Amazon Inc.[2]
{roy206, bbshail}@purdue.edu
{chengjio, phanide, smsarwar, chaosd, hhaiz}@amazon.com

Abstract

Personalized retrieval should reorder the *same* candidate set differently for different users: a query such as "comedy movies" should rank retrieved movies differently depending on a user's history and stated intent, not a single global ranking. We present a re-ranking framework that decomposes relevance into interpretable criteria (e.g. genre, actor-type, and quality for movies) and scores each candidate as a non-negative weighted sum of per-criterion matches, where the criterion importances are estimated separately from the user's watch history and from the query intent and then fused together. We introduce *criterion-level credit assignment* for Group Relative Policy Optimization (criterion-GRPO), which factors the group-relative advantage across criteria to provide a criteria-wise training signal than a list-level reward alone. We compare three different approach for scoring different criteria 1) LLM-estimated criteria importance, 2) Learned criteria importance 3) No-criteron scoring (Direct GRPO training). For the reward model, we experiment with ground-truth ranks whenever available or LLM-as-judge and metadata-matching based scoring. With extensive experimentation over Movielens and Amazon Reviews dataset, we observed that LLM-scored criteria importances achieve the highest NDCG@5 scores of 88-90% for personalized re-ranking of movies/items, while remaining fully interpretable, and an analysis of the learned importances shows they recover the intended query genre intent and user taste, giving faithful per-criterion explanations at no measured cost in accuracy.

1 Introduction

Given a query, a retrieval system returns a set of candidates; the role of a *re-ranker* is to order that set so the most relevant items appear first. When the system is personalized, relevance is not a global property of an item but a joint property of the item, the user, and the query. Consider the query "comedy movies" issued by two different users. A retrieval stage returns the same handful of comedies for both, but the desired ordering differs: a user whose history favors romantic, feel-good comedies should see those float to the top, while the movies from action and drama genre should rank first only for a user who prefers it. The query fixes the candidate *set*; the user's personalized watch history fixes the *order*. Figure 1 illustrates the setting.

Two requirements follow. The re-ranker must be *personalized* (sensitive to the user) and *intent-aware* (sensitive to the query). A third, increasingly important requirement is *interpretability*: as such systems are deployed in consequential settings, it is valuable to say *why* an item was ranked where it was, so that a user can inspect and audit.

A natural way to obtain personalized ranking is to decompose relevance into a small set of named criteria— for movies it is genres, actor-types, and overall quality—and to score a candidate as a weighted sum of how strongly it exhibits each criterion with respect to the personalized activity of the user and also the query intent. This reduces re-ranking to estimate, per user and per query, *how much each criterion matters*. The central question we study is where those criterion importances should come from. We compare three different approaches to estimate the importance of criterion with respect to user and query: (i) importances supplied directly by an LLM prompted with the user history and the query; (ii) importances learned by trainable heads over frozen text embeddings; and (iii) a criterion-free baseline in which a language model reads a candidate and emits a score directly, with no decomposition. For the LLM-scored importance, LLM's semantic understanding capability provide importance score for different criteria for each retrieved candidate. based on user's personal-

[*]Corresponding Author
- Work done as an intern at Amazon

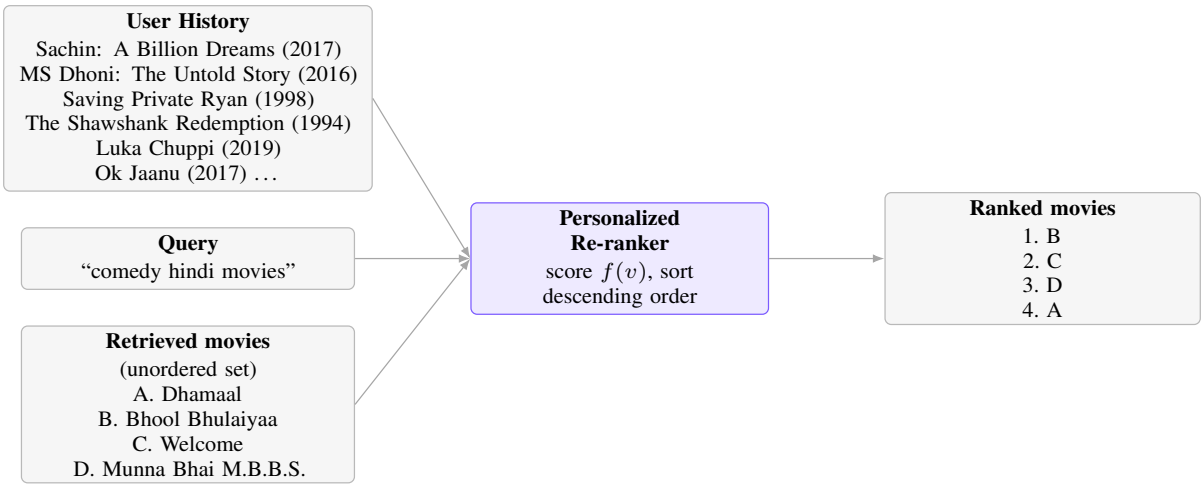


Figure 1: Personalized re-ranking task example. Given a user's history, a query, and a *unordered* retrieved candidate set, the personalized re-ranker outputs an ordered list. The query fixes the candidate *set*; the user preferences reranks the *order* of candidates: here B and C rise for a user whose history favors romantic comedies, while the globally highest-rated comedy movie A stays last as it doesn't have romantic angle.

ized history and query intent and for the learned criteria the criteria scores are learned in latent space from user's personalized description, query and candidates. The third approach is no-criterion version where the decomposition into criterion is not used rather direct scoring of the candidates are performed by using an LLM. We claim that the criterion-GRPO provides more flexibility in ranking rather than using SFT with NDCG@5 score for two reasons. First, criterion-GRPO provides control to rank movies based on different criteria by controling the criterion importance score and Second, criterion-GRPO can provide interpretability analysis of different items based on different criteria.

To train these scorers we use reinforcement learning with group-relative policy optimization as our objective to optimize the model parameters. For the reward model, we either use ground truth ranking (movie rating/product rating) when available. We also experiment with LLM-as-judge or metadata matching. For LLM-as-judge, we obtain graded labels from an LLM judge that reads the user history, the query, and all candidates and returns a full ground-truth ranking, which we cache and for metadata matching we use a weighted sum of different metadata match e.g. genre-match, actor-type match and quality for Movielens dataset. We optimize the scorer with Group Relative Policy Optimization (GRPO; Shao et al., 2024) and introduce a *criterion-level credit assignment* scheme (criterion-GRPO) that factors the group-relative advantage across criteria, giving a criterion-based learning signal than the list-level reward in rank-GRPO (Zhu et al., 2026). With extensive experiments in two different dataset named MovieLens and Amazon Reviews our approach criterion-GRPO, achieves the best scores when the criterion are scored with LLMs compared with learned criterion. Additionally, we show how the criterion scoring provides interoperability of ranking of different items by showing different criterion scores.

Our contributition in this work are as follows.

- We propose a personalized re-ranker that estimates criterion-decomposed scores from query intent and personalized user activities and fuses them to score each candidate and produce a personalized rankings for the retrieved items.
- We introduce criterion-based Group Relative Policy Optimization (criterion-GRPO) where the credits are assigned for different criteria with a criterion-based learning signal in GRPO objective.
- Our experimentation suggests that LLM-judged criterion performs better than learned criterion or no-criterion version while ranking movies and items in Movielens and Amazon Reviews dataset respectively.
- We present an interpretability analysis showing the learned importances recover the intended intent of query and supports user's personalized preferences.

2 Related Work

Learning to rank. Ranking has been studied through pointwise, pairwise, and listwise objectives (Liu, 2009). Listwise methods such as ListNet (Cao et al., 2007) and ListMLE (Xia et al., 2008) model a distribution over permutations, typically via the Plackett–Luce model (Plackett, 1975; Luce, 1959). Because the target metric NDCG (Järvelin and Kekäläinen, 2002) is non-differentiable, a body of work optimizes smooth surrogates: LambdaRank weights pairwise updates by the NDCG change from a swap (Burges, 2010), and ApproxNDCG differentiates NDCG through soft ranks (Qin et al., 2010). We adopt a Plackett–Luce policy and use both a reinforcement-learning objective and these supervised surrogates.

Personalized Re-ranking. Cross-encoders that jointly read query and candidate are the standard for re-ranking (Nogueira and Cho, 2019), while personalization draws on user history and behavior (**?**Bennett et al., 2012). Our no-criterion baseline is a personalized cross-encoder; our criterion-based models add an interpretable decomposition on top, trading some capacity for explanations.

RL from feedback and LLM judges. Policy-gradient training from a reward model underlies RLHF (Ouyang et al., 2022; Schulman et al., 2017); GRPO (Shao et al., 2024) removes the value network by estimating advantages from a group of samples. Using a strong LLM to produce evaluation or preference labels—LLM-as-a-judge—has become common (Zheng et al., 2023). We use an LLM judge to produce *full rankings*, not just pairwise preferences, and cache them as the reward. For parameter efficiency we use LoRA (Hu et al., 2022) over a Qwen3 backbone (Yang et al., 2025). Unlike prior work that treats the LLM only as an evaluator, we additionally use an LLM to supply interpretable per-criterion importances, and study when that prior beats a learned representation.

Interpretable and aspect-based ranking. A recurring goal is to explain *why* an item is ranked as it is, either by decomposing relevance into named aspects or by attributing a score to interpretable features. Our decomposition is a linear-in-criteria scorer whose per-criterion contributions are directly readable, placing the interpretability in the model rather than in a post-hoc attribution. In contrast to a monolithic cross-encoder, whose score is opaque, the importances W are themselves the explanation; §5.3 tests whether they are *faithful* by checking that they recover the benchmark's known intent and taste structure.

3 Problem Formulation

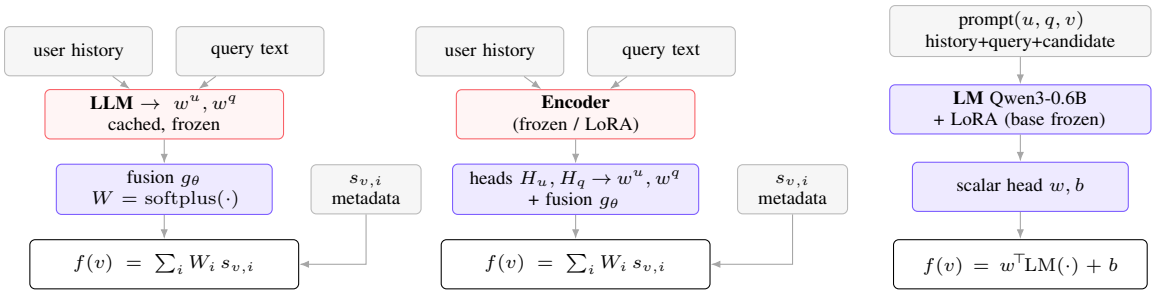
For a query q issued by user u, a retrieval stage returns a candidate set $\mathcal{V}_q = \{v_1, \ldots, v_K\}$ e.g. $K = 20$. Given the query q from the user u, the retrieved candidate set $\mathcal{V}_q$ and the user activity $\mathcal{H}_u$, the problem of personalized re-ranking aims to produces a permutation π of $\mathcal{V}_q$ that places the most relevant candidates first, where relevance is personalized (depends on $\mathcal{H}_u$) and intent-aware (depends on q). Each candidate $v \in \mathcal{V}_q$ is assigned a scalar score $f(v)$ and sorts in descending order; and the ranking models learns $f(v)$. We evaluate ordering quality with NDCG@5 on held-out queries.

4 Methodology

In this section, we describe the idea of criterion-decomposition of each candidate for personalized re-ranking supporting query intent and user persona. Next, we present three different approach for scoring criteria LLM-judged, learned criteria and criterion-Free approach. We also elaborate how we can either use ground truth ranking as the reward model or LLM-as-judge to rank candidates. Also, meta-data matching between candidate and user history/query is presented as an alternative of ground-truth ranking of candidates. Finally, we present how the criterion-decomposition can provide criterion-level signal in Group Relative Policy Optimization yielding to criterion-GRPO, an interpretable approach for personalized query oriented ranking.

4.1 Criterion decomposition

For each candidate retrieved from the query by a retriever, we decompose it into N different interpretable criterion for ranking. Those criteria can be task-specific and determine the relevance of the candidate in different aspects with respect to the query and the user's personalized interest. For example, for the Movielens dataset relevance is decomposed over $N = 13$ interpretable criteria: six genres, six actor-types, and quality. For a candidate v, a criterion score $s_{v,i} \in [0, 1]$ records how strongly v exhibits criterion i estimating from the metadata (genre membership $\rightarrow 1.0/0.1$; actor-type $\rightarrow 1.0/0.0$; quality $\rightarrow$ IMDB$/10$).



(a) LLM-scored criterion (b) Learned criterion (c) No criterion

Figure 2: The three re-ranking architectures. Pink denotes weights are frozen or LLM-supplied; violet nodes are finetuned with GRPO. (a) and (b) includes the criterion-decomposed score $f(v) = \sum_i W_i s_{v,i}$ and differ only in how the criterion importances W are produced; (c) removes the decomposition and scores each candidate directly.

Let $W_i \geq 0$ be the criterion importance parameter for criterion i and $s_{v,i}$ is the score for a candidate obtained by matching metadata. A candidate's contribution and score are

$$f(v) = \sum_{i=1}^{N} W_i\, s_{v,i}. \tag{1}$$

We aim to learn the criterion importance parameter W_i with respect to the user interest and query intent. We estimate importance separately from the user side and the query side, reflecting an asymmetry in the task: genre preference is largely query-driven, whereas actor-type and quality preference are user-driven. Given a user-side vector $w^u \in [0,1]^N$ and a query-side vector $w^q \in [0,1]^N$, we learn a per-criterion fusion network $g_\theta : \mathbb{R}^2 \to \mathbb{R}$ that produces the criterion importance score.

$$W_i = \mathrm{softplus}\big(g_\theta(w_i^u,\, w_i^q)\big). \tag{2}$$

4.2 Three sources of criterion importance

To learn the criterion importance score with respect to w_u and w_q, we present three different approach a) LLM-scored criterion importance b) Learned criterion importance c) No-Criterion. The approaches differ only in how w^u, w^q are produced; Figure 2 summarizes them.

LLM-scored criterion. An LLM is prompted with the user's watch history to obtain w^u and with the query text for w^q and returns a weight in $[0,1]$ for each named criterion. The choice of predefined criterion help LLMs to use the semantic capability to perform criterion scoring. These are cached once per user and per query and held fixed; only the fusion network g_θ is trained.

Learned criterion. The user history and query text are embedded by a frozen sentence encoder, and trainable heads H_u, H_q produce $w^u = \sigma(H_u e_u)$, $w^q = \sigma(H_q e_q)$. The heads and fusion network are trained; the encoder may be optimized LoRA-adapted, where the embeddings can no longer be cached and text is encoded on the fly.

No criterion. We do not decompose into criteria for this variant and this variant resembles state-of-the-art approach rank-GRPO (Zhu et al., 2026) for ranking documents optimized by GRPO objective. A encoder prompt containing the user history, the query, and a single candidate is scored directly by a language model (Qwen3-0.6B with LoRA over a frozen base) plus a scalar head: $f(v) = w^\top \mathrm{LM}(\mathrm{prompt}(u,q,v)) + b$. This model reads the candidate description and can in principle express relevance factors the fixed criteria cannot, at the cost of one forward pass per candidate and far more trainable parameters.

4.3 Reward Model

- **Ground Truth**: We can either use the ground-truth score e.g. movie ratings/product ratings and sort them and use the sorted order as the ground truth ranking as a reward model and to evaluate our models.
- **Metadata-matching**: Whenever ground truth scores are not available, we can use metadata-matching as a means of scoring each candidate and sort the score to provide a pseudo-ranking which can serve as a reward model and also help in evaluation of model performance.
- **LLM-as-judge**: Instead of a hand-written relevance formula, we can obtain labels from an LLM judge. For each query the judge reads

```
Judge (system).
You rank movies for a specific user's search
query.  Order ALL candidates from most to least
relevant for THIS user and THIS query.  Weigh:
(1) match to the query intent, (2) the user's
tastes from their history, (3) overall quality.
Reply ONLY with JSON {"ranking":["id1",...]}
listing every candidate id once, best first.
Judge (user, real example).
User's watch history:  Interstellar (drama;
imdb 8.7); No Country for Old Men (thriller,
drama; imdb 8.2); Sholay (action, drama)...
Query:  edge-of-your-seat thrillers
Candidates:
[hw47] Casino Royale (2006) - genres:  action,
  thriller; imdb 8.0; cast:  Daniel Craig ...
[hw11] The Godfather (1972) - genres:  drama;
...
```

```
Importance (user-side, system).
Give an importance weight in [0,1] for each
criterion for THIS user from their history.
Reply ONLY JSON criterion -> weight.  Criteria:
  - drama:  the drama genre
  - action_star:  marquee action star
  - quality:  critically acclaimed / rated
  ...  (13 total)
Reply.
{"drama":0.8, "thriller":0.9, ...,
  "action_star":0.6, "quality":0.9}
```

Figure 3: Prompts with real text and example. Top: the judge that produces the cached ground-truth ranking used as the reward. Bottom: the importance prompt used by the LLM-scored variant, issued separately for the user history (w^u) and the query (w^q).

the user history, the query, and all K candidates (title, genres, rating, cast, description) and returns a full ranking , best to worst (Figure 3, top). A defensive parser strips code fences, keeps only valid candidate ids, removes duplicates, and appends any omitted candidates, yielding a complete permutation. Rankings are cached (100 calls total, written incrementally), so training makes no further API calls and every approach is scored against identical labels.

We derive graded labels y_v from the judged position,

$$y_v = 1 - \frac{\text{rank}(v) - 1}{K - 1}, \tag{3}$$

so the judge's top pick receives $y = 1$ and its last $y = 0$. The reward of an ordering is the rank-discounted (DCG) aggregate

$$R(\pi) = \sum_{v \in \mathcal{V}_q} \text{disc}\big(\text{rank}_\pi(v)\big)\, y_v, \tag{4}$$

with $\text{disc}(j) = 1/\log_2(j+1)$.

4.4 Ranking policy and criterion-GRPO

We turn scores into probabilities using stochastic Plackett–Luce policy,

$$P_\theta(\pi) = \prod_{j=1}^{K} \frac{\exp f(\pi_j)}{\sum_{l=j}^{K} \exp f(\pi_l)}, \tag{5}$$

sampled with Gumbel noise. To apply Group Relative Policy Optimization objective (Shao et al., 2024) to optimize the model parameters, for each query we draw G orderings and score each with the reward model Eq. (4). The rank-GRPO approach standardizes the advantage within the group,

$$A^{(g)} = \frac{R(\pi^{(g)}) - \mu_R}{\sigma_R + \varepsilon}, \tag{6}$$

Criterion-level credit assignment. For criterion-based models we factor the reward across criteria. Let $\rho_{v,i}$ be criterion i's share of candidate v's score.

$$\rho_{v,i} = \frac{c_{v,i}}{\sum_j c_{v,j}} \tag{7}$$

This guides the reward through these criterion shares and provides a per-criterion reward

$$R_i(\pi) = \sum_v \rho_{v,i}\, \text{disc}(\text{rank}_\pi(v))\, y_v \tag{8}$$

where $\sum_i R_i = R$.

Group-standardizing R_i and removing the cross-criterion mean gives a mean-zero per-criterion advantage, and the combined advantage used in the update is

$$\tilde{A}_i^{(g)} = \frac{R_i(\pi^{(g)}) - \mu_i}{\sigma + \varepsilon}, \quad \bar{A}_i^{(g)} = \tilde{A}_i^{(g)} - \tfrac{1}{N}\sum_k \tilde{A}_k^{(g)}, \tag{9}$$

$$A_i^{(g)} = \underbrace{A^{(g)}}_{\text{rank-GRPO}} + \lambda \underbrace{\bar{A}_i^{(g)}}_{\text{criterion-based credit Assignment}}. \tag{10}$$

Setting $\lambda = 0$ recovers plain list-level GRPO; the criterion-level variant uses $\lambda = 0.3$. Here μ_i and σ are the mean and standard deviation of reward scores over different orderings in G.

We present the training algorithm for criterion-GRPO in Algorithm 1.

Algorithm 1 criterion-GRPO Training

Precompute: labels y (3);
LLM-based Criterion: cache w^u, w^q from LLM;
Learned Criterion: cache text embeddings.
Initialize trainable parameters θ; snapshot reference scores f_{ref} per query.
for each epoch **do**
  for each query $q \in \mathcal{Q}_{\text{tr}}$ **do**
    Compute scores $f(v)$: criterion-based (1) or directly for no-criterion.
    Sample $\pi^{(1)}, \ldots, \pi^{(G)} \sim P_\theta$; compute $R(\pi^{(g)})$ and list advantage (6)
    if criterion-based and criterion-level **then** compute R_i, $\bar{A}_i^{(g)}$; combine (9)
    end if
    Optimize Model Parameters with KL penalty.
  end for
end for
Evaluate: rank each $q \in \mathcal{Q}_{\text{ev}}$ by f; report mean $NDCG@5$.

4.5 Supervised Finetuning

Instead of using criterion-based decomposition and apply GRPO to finetune the personalized LLM-rankers, we can also optimize the model parameters by using Supervised Finetuning (SFT) by optimizing the NDCG@5 score between the predicted ranking and the ground truth ranking provided by the reward model. However, this approach doesn't allow the flexibity to control the ranking based on different criteria and also lacks interpretability.

4.6 Criterion-based Interpretable Ranking

Since we adopt the criterion-based decomposition for each candidate, it allows us to interpret how each candidate is relevant to different criteria use the criterion-based importance scores. We present experimental results based on the criterion-importance scores in Section 5.3

5 Experimental Results

Dataset Description. We conduct experiments on two different datasets namely MovieLens and Amazon Reviews dataset. We use a subset of the original datasets. The MovieLens dataset is basically consists of reviews for English and Hindi movies. For this dataset, we select 500 real movies. Then, we generate 500 users from 6 different user profiles (Drama Fan, Sports Fan, Comedy Fan etc.).

Dataset Name	Component	Count
MovieLens	Movies (real metadata)	500
	User Profiles	6
	Users	500
	Queries (train/eval)	10000 (8000/2000)
	Candidates Per Query	20
	Criteria (Genre / Actor Type / Quality)	13
	Ground Truth	IMDB / Rotten Tomatoes
Amazon Reviews	Products	2000
	User Profiles	10
	Users	200
	Queries	2000 (1600/400)
	Candidates Per Query	20
	Criteria (Product Type / Brand / Quality)	18
	Ground Truth	Product Rating

Table 1: Dataset Description

Later, for each user we create 20 different queries resulting into 10000 queries. We use 80:20 ratio to split the queries into training and evaluation set. We have IMDB/Rotten Tomatoes scores of different movies which can be used to sort the movies. Also, we can use LLM-as-judge/Metadata-based reward model to be used as the ground truth ranking. For the Amazon Reviews dataset, we select 200 products and sample 200 users from 10 different user profiles (Electronic Consumer, Furniture Consumer etc.). After that, we produce 2000 queries and split them into training and test set containing 1600 and 400 queries. For the ground truth ranking, this dataset have product rating which can be sorted to obtain the ground truth ranking. We provide the description of dataset in Table 1.

Models and training. For the reward model, we have three different options: we can either use the ground truth ranking whenever available. If the ground truth ranking is not available then we can either use LLM-as-judge to rank the candidates or the metadata-match based score (genre-type, actor-type, movie qulaity). The LLM judge and the LLM importance estimator are Claude (Opus-5), accessed once per item and cached. The learned-criterion encoder is Qwen3-Embedding-0.6B; the no-criterion reranker is Qwen3-0.6B (Yang et al., 2025) adapted with LoRA (Hu et al., 2022) (r=8) over a frozen base, with a scalar head. We train with GRPO (G=16, KL coefficient 0.02) at the list level (λ=0) and the criterion level (λ=0.3), and report NDCG@5 on the held-out queries.

5.1 Performance Analysis

In Table 2, we present the ranking performance of LLM-based Criterion, Learned-Criterion and No-Criterion approaches on Movie Lens and Amazon Reviews dataset. For the ground truth ranking, we experimented with LLM-as-judge to rank the candidates, used real labels (IMDB/Rotten Tomatoes for

Dataset	Model Type	Ground Truth		
		LLM-as-judge	Real Labels	Metadata-matching
Movie Lens	LLM-based Criterion	**0.88**	**0.92**	**0.95**
	Learned Criterion	0.85	0.90	0.91
	No-Criterion (Rank-GRPO)	0.80	0.85	0.83
Amazon Reviews	LLM-based Criterion	**0.89**	**0.90**	**0.92**
	Learned Criterion	0.88	0.88	0.90
	No-Criterion (Rank-GRPO)	0.85	0.83	0.86

Table 2: Performance comparison of LLM-based criterion, learned criterion and no-criterion approaches on Movielens and Amazon Reviews dataset. NDGC@5 scores are reported and the best performances are marked in bold.

MovieLens, product ratings for Amazon Reviews), and the metadata-match score. The ordering is consistent across *every* dataset and ground-truth column: **LLM-based criterion** $>$ **learned criterion** $>$ **no-criterion**. On MovieLens the LLM-based criterion attains $0.88/0.92/0.95$, ahead of the learned criterion ($0.85/0.90/0.91$) and the no-criterion cross-encoder ($0.80/0.85/0.83$); on Amazon Reviews the same order holds ($0.89/0.90/0.92$ vs. $0.88/0.88/0.90$ vs. $0.85/0.83/0.86$).

LLM-based criteiron scoring surpass learned criterion and no-criterion version: For the LLM-based criterion we can observe superior ranking performance compared to the learned criterion version where the weights of different criteria are learned and the no-criterion version where the criterion decomposition is not applied. The No-Criterion version is very close to state-of-the-art approach rank-GRPO which directly use the ground truth ranking as the reward and use the GRPO objective to optimize the parameters. In all cases, however, the criterion-based models outperform the criterion-free reranker, indicating that the decomposition is a net benefit for both ranking quality and (as shown in §5.3) interpretability. The criterion-decomposition achieves better NDGC@5 score as it aims to grade the candidate videos on different criteria e.g. genre for Movie Lens dataset and product type for Amazon Reviews dataset. Based on these importance scores and the match between the metadata the candidate videos or products are provided a score which is sorted to give the predicted ranking. Since LLMs are better at understanding the semantic between the user preferred genre/product type and the candidate movie/product's genre/-type it can provide better criterion weight which lead to better performance for the LLM-based criterion version. the criterion decomposition helps most when

Metadata-matching provides better scores for the ranking: the criterion decomposition helps most when relevance is largely explained by the criteria: the metadata-based reward, which is itself a function of genre/actor/quality matches, yields the highest scores for the criterion-based approaches (0.95 on MovieLens), whereas the no-criterion model, which cannot exploit that structure, gains far less. Second, the gap between the LLM-supplied and learned importances are dataset-dependent: it is wide on MovieLens (0.02–0.04 NDCG@5) but narrows on Amazon Reviews (0.01–0.02). We attribute this to metadata quality—the Amazon criterion scores are noisier and the actor-type criteria are inactive, so the advantage of a strong LLM prior over learned importances shrinks when the underlying signal the prior encodes is weaker.

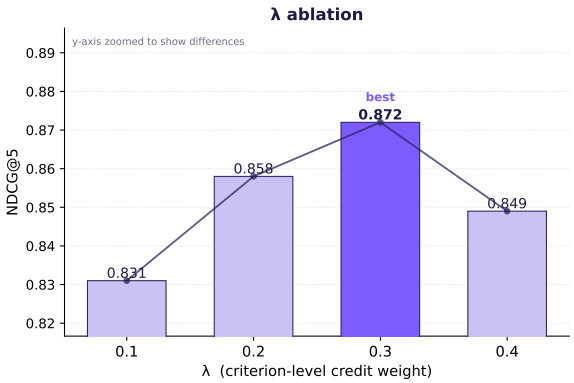


Figure 4: Ablation Experiment on the impact of criterion-level credit weight, λ

5.2 Ablation Studies

Effect of the criterion-level credit weight λ. The combined advantage in Eq. (9) interpolates between list-level GRPO (λ=0) and criterion-level credit assignment. We sweep $\lambda \in$

$\{0.1, 0.2, 0.3, 0.4\}$ on the LLM-based criterion model and report held-out NDCG@5 in Figure 4. Performance rises monotonically as λ increases from 0.1 (0.831) through 0.2 (0.858) to a peak at 0.3 (0.872), then declines at 0.4 (0.849). This interior optimum matches the intended role of the term: a small amount of per-criterion shaping supplies a denser learning signal than the list-level reward alone, but too large a weight lets the criterion-level term dominate the list objective and adds variance, eroding the gain. The peak at 0.3 motivates our choice of λ=0.3 in all main experiments. We note the differences across λ are modest (within roughly 0.04 NDCG@5); the trend is consistent, but a multi-seed study would be needed to establish the margins as statistically significant.

5.3 Interpretability of Ranking

Because the score is a sum of named terms, $f(v) = \sum_i W_i\, s(v, i)$, every ranking decision decomposes into per-criterion contributions $c_i = W_i\, s(v, i)$ that state, in words, why a candidate scored as it did. This attribution is *exact*, not a post-hoc approximation: the contributions are the actual arithmetic that produced the score. It is available for the LLM-based and learned-criterion models and is precisely what the no-criterion reranker cannot provide.

Figure 5 visualizes this for the held-out query *"powerful dramas"* (a drama-fan user), showing each top-ranked candidate as a stacked bar of its criterion contributions. The decomposition makes the model's behavior legible: for this drama-intent query the *drama* (≈36%) and *dramatic_lead* (≈32%) criteria dominate every candidate's score, with *quality* contributing a further ≈27%, confirming that the fused importances concentrate on the criteria the query and user care about. The top-ranked item, *Se7en*, is placed first by both our model and the LLM judge, and its bar shows why—the highest combined drama, dramatic-lead and quality contributions, plus a small *thriller* term (≈5%) that the other dramas lack.

The figure also surfaces an advantage of the decomposition, which we regard as a strength of the interpretability tool. Within a single genre the genre and actor-type contributions are identical across candidates (all dramas receive the same 46 and 42), so the ranking among them is decided almost entirely by the small *quality* term. This within-genre indiscriminability explains the divergences from the LLM judge lower in the list—for example, *Drishyam* is ranked #2 by our model but

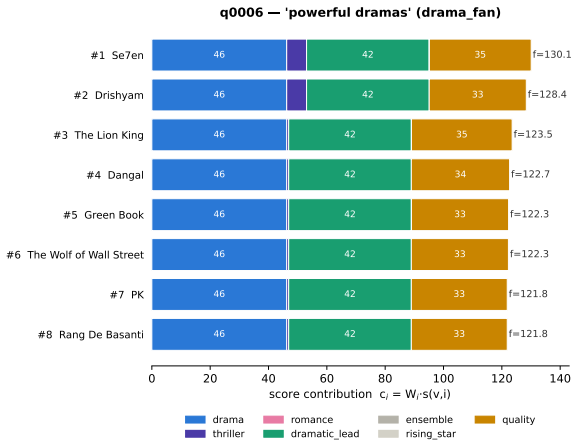


Figure 5: Interpretability of rankings from the criterion-based weights for the query "powerful dramas" for a drama fan user from the Movie Lens dataset. The decomposition of each candidate's score $c_i = W_i\, s(v, i)$ into named per-criterion contributions.

#10 by the judge, and *Rang De Basanti* #8 by our model but #3 by the judge— because the judge weighs finer, description-level distinctions that the fixed criteria cannot express. The value of the visualization is exactly that it makes this behavior explicit and auditable: one can read off that these items were separated only by a fraction of a point of quality, which no black-box reranker would reveal.

6 Conclusion

In this work, We study the problem of personalized reranking with criterion-decomposition. We present analysis where the criterion importances in an interpretable, personalized re-ranker should come from, comparing LLM-supplied importances, learned importances, and a criterion-free ranker. For the reward model, we adopt ground truth ranking when available or LLM-as-judge reward or metadata-matching based scoring. Our experiments over MovieLens and Amazon review datasets suggest that the LLM-supplied criterion importances provides the best personalized ranking NDCG@5 scores while remaining fully interpretable, and an analysis of the learned weights shows they recover the intended query intent and personalized to the user's watch history or purchasing pattern, so the explanations are faithful. The criterion-based decomposition allows to provide interpretability of the ranked items, benefiting more than the no-criterion version or Supervised Fine-tuning with differentiable NDCG@5 loss.

References

Paul N. Bennett, Ryen W. White, Wei Chu, Susan T. Dumais, Peter Bailey, Fedor Borisyuk, and Xiaoyuan Cui. 2012. Modeling the impact of short- and long-term behavior on search personalization. In *Proceedings of the 35th International ACM SIGIR Conference*, pages 185–194.

Christopher J.C. Burges. 2010. From RankNet to LambdaRank to LambdaMART: An overview. Technical Report MSR-TR-2010-82, Microsoft Research.

Zhe Cao, Tao Qin, Tie-Yan Liu, Ming-Feng Tsai, and Hang Li. 2007. Learning to rank: from pairwise approach to listwise approach. In *Proceedings of the 24th International Conference on Machine Learning*, pages 129–136.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Kalervo Järvelin and Jaana Kekäläinen. 2002. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446.

Tie-Yan Liu. 2009. Learning to rank for information retrieval. *Foundations and Trends in Information Retrieval*, 3(3):225–331.

R. Duncan Luce. 1959. *Individual Choice Behavior: A Theoretical Analysis*. Wiley.

Rodrigo Nogueira and Kyunghyun Cho. 2019. Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744.

Robin L. Plackett. 1975. The analysis of permutations. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 24(2):193–202.

Tao Qin, Tie-Yan Liu, and Hang Li. 2010. A general approximation framework for direct optimization of information retrieval measures. *Information Retrieval*, 13(4):375–397.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y.K. Li, Yu Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.

Fen Xia, Tie-Yan Liu, Jue Wang, Wensheng Zhang, and Hang Li. 2008. Listwise approach to learning to rank: theory and algorithm. In *Proceedings of the 25th International Conference on Machine Learning*, pages 1192–1199.

An Yang et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36, pages 46595–46623.

Yaochen Zhu, Harald Steck, Dawen Liang, Yinhan He, Vito Ostuni, Jundong Li, and Nathan Kallus. 2026. Rank-grpo: Training llm-based conversational recommender systems with reinforcement learning. In *International Conference on Learning Representations*, volume 2026, pages 155483–155512.

A Appendix

A.1 LLM-as-judge Reward Example

Query, User History and LLM-as-judge (Claude Opus-5) ranked candidate movie names with genre, imdb score and reward.

Query	"edge-of-your-seat thrillers"	intent=thriller_fan			
User History	Interstellar (drama; imdb 8.7); Baahubali: The Beginning (action, drama; imdb 8.0); No Country for Old Men (thriller, drama; imdb 8.2); Sholay (action, drama; imdb 8.1); Chak De! India (sports, drama; imdb 8.2); Gladiator (action, drama; imdb 8.5); The Shawshank Redemption (drama; imdb 9.3); Rang De Basanti (drama; imdb 8.1); Article 15 (thriller, drama; imdb 8.0); Forrest Gump (drama, romance; imdb 8.8)				
	LLM-as-judge Rank	**Movie Name**	**Genre**	**imdb**	**reward (y)**
Candidates	1	Inception	action, thriller	8.8	1.000
	2	Casino Royale	action, thriller	8.0	0.947
	3	Tumbbad	thriller, drama	8.2	0.895
	4	Gangs of Wasseypur	action, drama	8.2	0.842
	5	Stree	comedy, thriller	7.5	0.789
	6	The Godfather	drama	9.2	0.737
	7	Django Unchained	action, drama	8.5	0.684
	8	RRR	action, drama	7.8	0.632
	9	Padmaavat	drama, action	7.0	0.579
	10	The Shawshank Redemption	drama	9.3	0.526
	11	Hera Pheri	comedy	8.2	0.474
	12	Bajrangi Bhaijaan	drama, comedy	8.0	0.421
	13	Rocky	sports, drama	8.1	0.368
	14	Lagaan	drama, sports	8.1	0.316
	15	Queen	comedy, drama	8.1	0.263
	16	Chak De! India	sports, drama	8.2	0.211
	17	Dilwale Dulhania Le Jayenge	romance, drama	8.0	0.158
	18	Rockstar	romance, drama	7.6	0.105
	19	Yeh Jawaani Hai Deewani	romance, comedy, drama	7.2	0.053
	20	Badhaai Ho	comedy, drama	7.8	0.000

A.2 Ranking Example : LLM-Scored Criterion Imporance

Query, user history and ranking results from LLM-score criterion Importance with movie names, genre, imdb score, reward and LLM-as-judge Rank.

Query	"edge-of-your-seat thrillers"	intent=thriller_fan | | NDCG@5 = 0.956				
User History	Interstellar (drama; imdb 8.7); Baahubali: The Beginning (action, drama; imdb 8.0); No Country for Old Men (thriller, drama; imdb 8.2); Sholay (action, drama; imdb 8.1); Chak De! India (sports, drama; imdb 8.2); Gladiator (action, drama; imdb 8.5); The Shawshank Redemption (drama; imdb 9.3); Rang De Basanti (drama; imdb 8.1); Article 15 (thriller, drama; imdb 8.0); Forrest Gump (drama, romance; imdb 8.8)					
	Rank	**Movie Name**	**Genre**	**imdb**	**reward (y)**	**LLM-as-judge Rank**
Candidates	1	Tumbbad (2018)	thriller, drama	8.2	23.771	3
	2	Inception (2010)	action, thriller	8.8	23.016	1
	3	Casino Royale (2006)	action, thriller	8.0	22.582	2
	4	Django Unchained (2012)	action, drama	8.5	19.161	7
	5	Gangs of Wasseypur (2012)	action, drama	8.2	18.999	4
	6	RRR (2022)	action, drama	7.8	18.782	8
	7	Padmaavat (2018)	drama, action	7.0	18.348	9
	8	Stree (2018)	comedy, thriller	7.5	17.641	5
	9	The Shawshank Redemption (1994)	drama	9.3	12.850	10
	10	The Godfather (1972)	drama	9.2	12.795	6
	11	Chak De! India (2007)	sports, drama	8.2	12.287	16
	12	Queen (2013)	comedy, drama	8.1	12.265	15
	13	Lagaan (2001)	drama, sports	8.1	12.233	14
	14	Rocky (1976)	sports, drama	8.1	12.233	13
	15	Dilwale Dulhania Le Jayenge (1995)	romance, drama	8.0	12.214	17
	16	Bajrangi Bhaijaan (2015)	drama, comedy	8.0	12.211	12
	17	Badhaai Ho (2018)	comedy, drama	7.8	12.102	20
	18	Rockstar (2011)	romance, drama	7.6	11.997	18
	19	Yeh Jawaani Hai Deewani (2013)	romance, comedy, drama	7.2	9.556	19
	20	Hera Pheri (2000)	comedy	8.2	6.502	11

A.3 Ranking Example : Learned Criterion Imporance

Query, user history and ranking results from Learned criterion Importance with movie names, genre, imdb score, reward and LLM-as-judge Rank.

Query	"edge-of-your-seat thrillers"	intent=thriller_fan | NDCG@5 = 0.968				
User History	Interstellar (drama; imdb 8.7); Baahubali: The Beginning (action, drama; imdb 8.0); No Country for Old Men (thriller, drama; imdb 8.2); Sholay (action, drama; imdb 8.1); Chak De! India (sports, drama; imdb 8.2); Gladiator (action, drama; imdb 8.5); The Shawshank Redemption (drama; imdb 9.3); Rang De Basanti (drama; imdb 8.1); Article 15 (thriller, drama; imdb 8.0); Forrest Gump (drama, romance; imdb 8.8)					
	Rank	**Movie Name**	**Genre**	**imdb**	**reward (y)**	**LLM-as-judge Rank**
Candidates	1	Inception (2010)	action, thriller	8.8	21.379	1
	2	Tumbbad (2018)	thriller, drama	8.2	21.190	3
	3	Casino Royale (2006)	action, thriller	8.0	20.495	2
	4	Stree (2018)	comedy, thriller	7.5	19.417	5
	5	Django Unchained (2012)	action, drama	8.5	15.786	7
	6	Gangs of Wasseypur (2012)	action, drama	8.2	15.454	4
	7	RRR (2022)	action, drama	7.8	15.012	8
	8	Padmaavat (2018)	drama, action	7.0	14.128	9
	9	The Shawshank Redemption (1994)	drama	9.3	12.155	10
	10	The Godfather (1972)	drama	9.2	12.045	6
	11	Chak De! India (2007)	sports, drama	8.2	11.621	16
	12	Lagaan (2001)	drama, sports	8.1	11.510	14
	13	Rocky (1976)	sports, drama	8.1	11.510	13
	14	Queen (2013)	comedy, drama	8.1	10.861	15
	15	Dilwale Dulhania Le Jayenge (1995)	romance, drama	8.0	10.756	17
	16	Bajrangi Bhaijaan (2015)	drama, comedy	8.0	10.750	12
	17	Badhaai Ho (2018)	comedy, drama	7.8	10.529	20
	18	Rockstar (2011)	romance, drama	7.6	10.314	18
	19	Hera Pheri (2000)	comedy	8.2	9.940	11
	20	Yeh Jawaani Hai Deewani (2013)	romance, comedy, drama	7.2	8.898	19

A.4 Ranking Example : Criterion-Free Rank-GRPO

Query, user history and ranking results from Rank-GRPO with movie names, genre, imdb score, reward and LLM-as-judge Rank.

Query	"edge-of-your-seat thrillers"	intent=thriller_fan | | NDCG@5 = 0.756				
User History	Interstellar (drama; imdb 8.7); Baahubali: The Beginning (action, drama; imdb 8.0); No Country for Old Men (thriller, drama; imdb 8.2); Sholay (action, drama; imdb 8.1); Chak De! India (sports, drama; imdb 8.2); Gladiator (action, drama; imdb 8.5); The Shawshank Redemption (drama; imdb 9.3); Rang De Basanti (drama; imdb 8.1); Article 15 (thriller, drama; imdb 8.0); Forrest Gump (drama, romance; imdb 8.8)					
	Rank	**Movie Name**	**Genre**	**imdb**	**reward (y)**	**LLM-as-judge Rank**
Candidates	1	Gangs of Wasseypur (2012)	action, drama	8.2	0.147	4
	2	Badhaai Ho (2018)	comedy, drama	7.8	0.147	20
	3	Hera Pheri (2000)	comedy	8.2	0.147	11
	4	Tumbbad (2018)	thriller, drama	8.2	0.147	3
	5	Inception (2010)	action, thriller	8.8	0.147	1
	6	The Shawshank Redemption (1994)	drama	9.3	0.147	10
	7	Chak De! India (2007)	sports, drama	8.2	0.147	16
	8	Django Unchained (2012)	action, drama	8.5	0.147	7
	9	Queen (2013)	comedy, drama	8.1	0.147	15
	10	Lagaan (2001)	drama, sports	8.1	0.147	14
	11	Casino Royale (2006)	action, thriller	8.0	0.147	2
	12	RRR (2022)	action, drama	7.8	0.147	8
	13	Yeh Jawaani Hai Deewani (2013)	romance, comedy, drama	7.2	0.147	19
	14	Padmaavat (2018)	drama, action	7.0	0.147	9
	15	Dilwale Dulhania Le Jayenge (1995)	romance, drama	8.0	0.147	17
	16	Rockstar (2011)	romance, drama	7.6	0.147	18
	17	Rocky (1976)	sports, drama	8.1	0.147	13
	18	Stree (2018)	comedy, thriller	7.5	0.147	5
	19	Bajrangi Bhaijaan (2015)	drama, comedy	8.0	0.147	12
	20	The Godfather (1972)	drama	9.2	0.147	6

A.5 Full Prompt Templates and Hyperparameters

The judge and importance prompts are abridged in Figure 3; the released code contains the complete templates, the JSON parser (fence stripping, id validation, de-duplication, and omission repair), and the candidate-rendering format. GRPO uses group size $G{=}16$, KL coefficient 0.02, Gumbel-sampled Plackett–Luce rollouts, and a per-query cached reference for the KL anchor. The fusion network is a two-layer MLP ($2 \to 16 \to 1$, $\tanh$) applied per criterion. The no-criterion reranker uses LoRA rank $r{=}8$ over a frozen Qwen3-0.6B, with left-side padding and truncation so that the candidate (placed at the end of the prompt) is preserved under last-token pooling.

A.6 Computational cost

The three approaches differ sharply in cost per training step. The criterion-based models operate on cached quantities: the LLM-scored variant caches w^u, w^q once per user and query, and the learned variant (with a frozen encoder) caches text embeddings; a training step then touches only the small fusion network and heads. The no-criterion cross-encoder instead runs the language model once per candidate, i.e. $K{=}20$ forward passes per query per step, and cannot cache candidate representations because they depend on the user and query. This gap—a handful of small matrix multiplies versus K transformer forward passes—compounds over rollouts under GRPO, where each of the G sampled orderings is scored from the same candidate scores. A bi-encoder that encodes each candidate once and scores by an inner product would recover cacheability at the cost of the cross-encoder's query–candidate interaction; we treat this as an orthogonal design axis and use the cross-encoder as the no-criterion representative.